

Rendu infrastructure et DevOps

Introduction + CI

Exercice 1

Une fois après avoir cloné le projet, lorsque l'on essaye de le lancer, on obtient une erreur due à l'absence de base de données. Il ne nous est donc pas possible d'effectuer une requête HTTP, le serveur n'étant pas lancé. Nous devons donc rajouter un service Postgres dans le docker-compose dans le futur.

```
rourou@LAPTOP-U6739NS7:~/esgi/docker-swarm-rendu/esgi-2603-my-favorite-places/server$ pnpm dev
> server@1.0.1 dev /home/rourou/esgi/docker-swarm-rendu/esgi-2603-my-favorite-places/server
> nodemon src/index.ts

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: ts,json
[nodemon] starting `ts-node src/index.ts`
⌚ starting server!
🔥 unable to connect to db: authentication par mot de passe échoué pour l'utilisateur postgres
[nodemon] clean exit - waiting for changes to be re-run
```

On peut cependant commencer à dockerizer le `/server` :

```
FROM node:24-alpine
WORKDIR /app

COPY package.json package.json
RUN npm i

COPY src src
COPY tsconfig.json tsconfig.json
CMD npm start
```

On lance alors un build: `docker build .`

```

rourou@LAPTOP-U6739NS7:~/esgi/docker-swarm-rendu/esgi-2603-my-favorite-places/server$ docker build .
[+] Building 41.0s (12/12) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 174B
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/node:24-alpine
=> [auth] library/node:pull token for registry-1.docker.io
=> [1/6] FROM docker.io/library/node:24-alpine@sha256:2bdb65ed1dab192432bc31c95f94155ca5ad7fc1392fb7eb7526ab682fa5bf14
=> => resolve docker.io/library/node:24-alpine@sha256:2bdb65ed1dab192432bc31c95f94155ca5ad7fc1392fb7eb7526ab682fa5bf14
=> => sha256:22114f5b47161a597998c48e339fac8ded77f7bba0a14c595e6a139afdf49cd8 6.52kB / 6.52kB
=> => sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb 3.86MB / 3.86MB
=> => sha256:eea2c8c31119c028a455d4bf33b2d3db30709fc3cbe5731a86fd418a2f2a047b 52.28MB / 52.28MB
=> => sha256:414f813165cf3485659f259590b5a84ca11c7caa28c508fff740ddb7f59f00ef 1.26MB / 1.26MB
=> => sha256:2bdb65ed1dab192432bc31c95f94155ca5ad7fc1392fb7eb7526ab682fa5bf14 3.87kB / 3.87kB
=> => sha256:9a2c6269f83d74af45665c0b738b78c7b5f07bf2374c72a4c540065a975b9693 1.72kB / 1.72kB
=> => sha256:c7fd89fde4162535dc23357092d0e79d83508e8176f3aee3aef4a3a20ec1f450 445B / 445B
=> => extracting sha256:6a0ac1617861a677b045b7ff88545213ec31c0ff08763195a70a4a5adda577bb
=> => extracting sha256:eea2c8c31119c028a455d4bf33b2d3db30709fc3cbe5731a86fd418a2f2a047b
=> => extracting sha256:414f813165cf3485659f259590b5a84ca11c7caa28c508fff740ddb7f59f00ef
=> => extracting sha256:c7fd89fde4162535dc23357092d0e79d83508e8176f3aee3aef4a3a20ec1f450
=> [internal] load build context
=> => transferring context: 11.72kB
=> [2/6] WORKDIR /app
=> [3/6] COPY package.json package.json
=> [4/6] RUN npm i
=> [5/6] COPY src src
=> [6/6] COPY tsconfig.json tsconfig.json
=> exporting to image
=> => exporting layers
=> => writing image sha256:9fe4c4961f7aa910d107ae12bae06884556b702bf5eaba0073ec845ed110c948

```

Nous allons maintenant créer un `docker-compose.yml` à la racine du projet. Nous allons y spécifier 2 services: le serveur et la base de données.

```

services:
  >Run Service
  server:
    build: ./server
    ports:
      - "3000:3000"
    depends_on:
      - db

  >Run Service
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: postgres
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: supersecret

```

Pour ce qui est de la base de données, il nous a fallu récupérer les informations de connexions situées dans `src/datasource.ts` :

```
const datasource = new DataSource({
  type: "postgres",
  host: "localhost",
  username: "postgres",
  password: "supersecret",
  database: "postgres",
  entities: [__dirname + "/entities/**/*.{js,ts}"],
  logging: true,
  synchronize: true,
});
```

Nous allons maintenant dockerizer le front-end React :

```
FROM node:24-alpine AS build
WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build
```

Et finalement, nous allons modifier le `docker-compose.yml` pour y ajouter notre nouveau service `client`, que l'on va mapper sur le port 4000:

```

1  services:
    ▷ Run Service
2  client:
3    build: ./client
4    ports:
5      - "4000:4000"
6    depends_on:
7      - server
8
9    ▷ Run Service
10 server:
11   build: ./server
12   ports:
13     - "3000:3000"
14   depends_on:
15     - db
16
17   ▷ Run Service
18 db:
19   image: postgres:17-alpine
20   environment:
21     POSTGRES_DB: postgres
22     POSTGRES_USER: postgres
23     POSTGRES_PASSWORD: supersecret

```

Afin de permettre la connexion, nous allons modifier dans le `client/vite.config.ts` la cible du proxy, pour y mettre le nom du service `server` :

```

5  export default defineConfig({
6    plugins: [react()],
7    server: {
8      proxy: {
9        "/api": {
10         target: "http://server:3000",
11         changeOrigin: true,
12       },
13     },
14   },
15 });
16

```

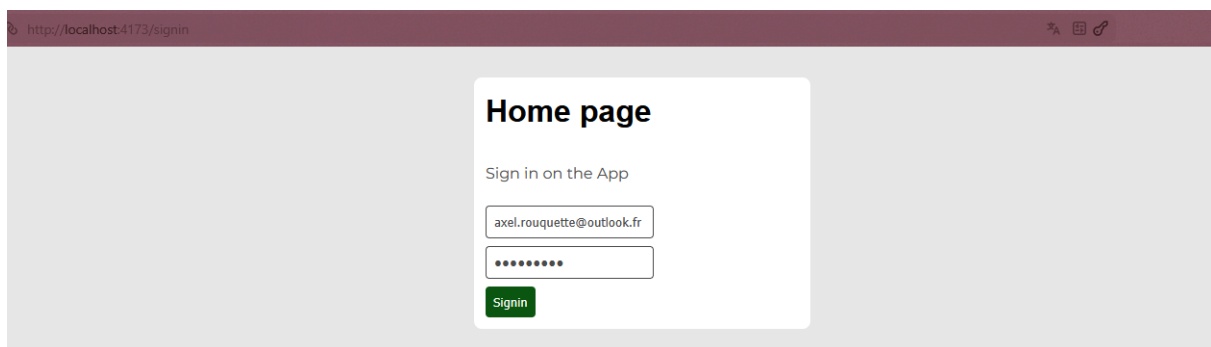
Ainsi que l'host de la `datasource.ts` ; nous allons y mettre le service `db` :

```

3  const datasource = new DataSource({
4    type: "postgres",
5    host: "db",
6    username: "postgres",
7    password: "supersecret",
8    database: "postgres",
9    entities: [__dirname + "/entities/**/*.{js,ts}"],
10    logging: true,
11    synchronize: true,
12  });
13

```

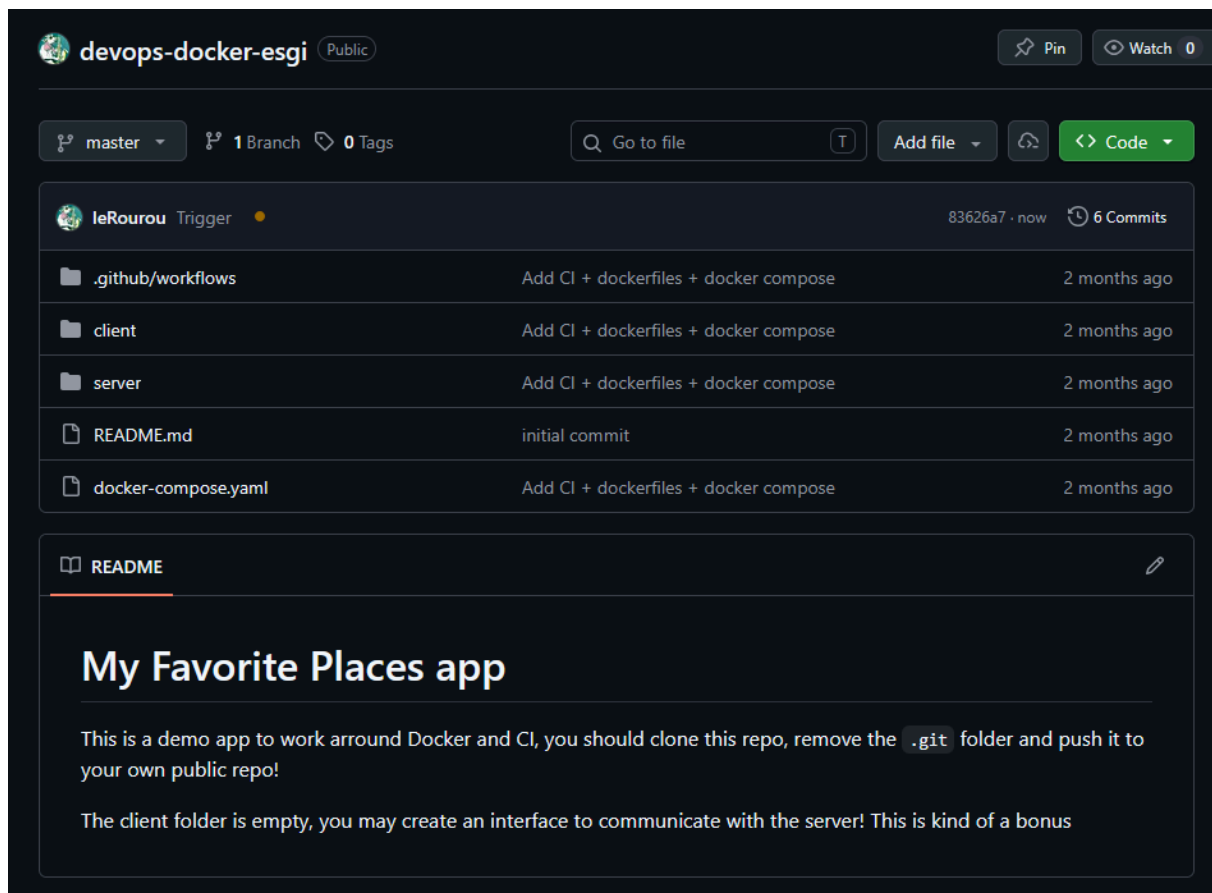
On peut enfin lancer le `docker-compose.yml` en utilisant la commande `docker-compose up --build`



Les services `db`, `server` et `client` communiquent ensemble grâce au protocole HTTP.

Exercice 3

On commence par créer le repository, puis on y ajoute un répertoire `.github/workflows`



On crée ensuite un fichier `build.yml` dans ce répertoire. On y spécifie qu'il doit se trigger à chaque push sur master. Pour ce qui est de la génération des jobs, on peut utiliser la documentation officielle. De mon côté, j'ai utilisé la documentation ainsi que l'IA pour débogger.

```
name: Build and push Docker images

on:
  push:
    branches:
      - master

env:
  REGISTRY: ghcr.io
  IMAGE_OWNER: ${ github.repository_owner }

jobs:
  build-and-push:
    runs-on: ubuntu-latest
```

```

permissions:
  contents: read
  packages: write
steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Set lowercase image owner
    run: echo "IMAGE_OWNER_LC=${IMAGE_OWNER,,}" >> $GIT
HUB_ENV
    env:
      IMAGE_OWNER: ${ github.repository_owner }

  - name: Log in to GitHub Container Registry
    uses: docker/login-action@v3
    with:
      registry: ${ env.REGISTRY }
      username: ${ github.actor }
      password: ${ secrets.GITHUB_TOKEN }

  - name: Build and push client image
    uses: docker/build-push-action@v6
    with:
      context: ./client
      push: true
      tags: ${ env.REGISTRY }/${ env.IMAGE_OWNER_LC
}}/my-favorite-places-client:latest

  - name: Build and push server image
    uses: docker/build-push-action@v6
    with:
      context: ./server
      push: true
      tags: ${ env.REGISTRY }/${ env.IMAGE_OWNER_LC
}}/my-favorite-places-server:latest

```

On va ajouter un test à `getDistance()` :

```
import { getDistance } from "../getDistance";
describe("getDistance", () => {
  it("returns 0 for the same point", () => {
    const point = { lat: 48.8566, lng: 2.3522 };
    expect(getDistance(point, point)).toBe(0);
  });
});
```

Ainsi qu'un job de test sur dans le `build.yml` du GitHub Actions :

```
12 jobs:
13   test:
14     runs-on: ubuntu-latest
15     steps:
16       - name: Checkout
17         uses: actions/checkout@v4
18
19       - name: Setup Node.js
20         uses: actions/setup-node@v4
21         with:
22           node-version: "22"
23           cache: "npm"
24           cache-dependency-path: server/package-lock.json
25
26       - name: Install dependencies
27         run: npm ci
28         working-directory: ./server
29
30       - name: Run tests
31         run: npm test
32         working-directory: ./server
33
```

Enfin, nous allons ajouter `Jest` aux dépendances du projet, et un script `npm run test` lançant la commande `jest`, permettant d'exécuter les tests :

```

{
  "name": "server",
  "version": "1.0.1",
  "main": "index.js",
  "license": "MIT",
  > Debug
  "scripts": {
    "build": "tsc",
    "start": "node dist/index.js",
    "dev": "nodemon src/index.ts",
    "test": "jest"
  },
  "dependencies": {
    "argon2": "^0.44.0",
    "axios": "^1.13.5",
    "express": "^5.2.1",

```

Après une session de débogage, on finit par avoir une pipeline qui fonctionne :

All workflows			
Showing runs from all workflows			
5 workflow runs Workflow ▾ Event ▾ Status ▾ Branch ▾ Actor ▾			
✓ Add jest types to tsconfig and update yarn.lock with new resolver bin... Build and push Docker images #5: Commit 3b27373 pushed by leRourou	master	3 minutes ago 2m 26s	...
✗ Refactor code structure for improved readability and maintainability Build and push Docker images #4: Commit e20f051 pushed by leRourou	master	11 minutes ago 1m 11s	...
✓ Set lowercase image owner and update image tags Build and push Docker images #3: Commit 488bb62 pushed by leRourou	master	24 minutes ago 58s	...
✗ Trigger Build and push Docker images #2: Commit 83626a7 pushed by leRourou	master	27 minutes ago 15s	...
✗ ci: trigger workflow Build and push Docker images #1: Commit c827d33 pushed by leRourou	master	Mar 5, 6:21 PM GMT+1 1m 23s	...

On ajoute une règle à `master` sur le repo, interdisant le push d'un commit.

Rulesets / Protect master Active

Ruleset created

Ruleset Name *

Protect master

Enforcement status

Active

Bypass list

+ Add bypass

Exempt roles, teams, agents, apps, and users from this ruleset by adding them to the bypass list.

Bypass list is empty

Target branches

Which branches should be matched?

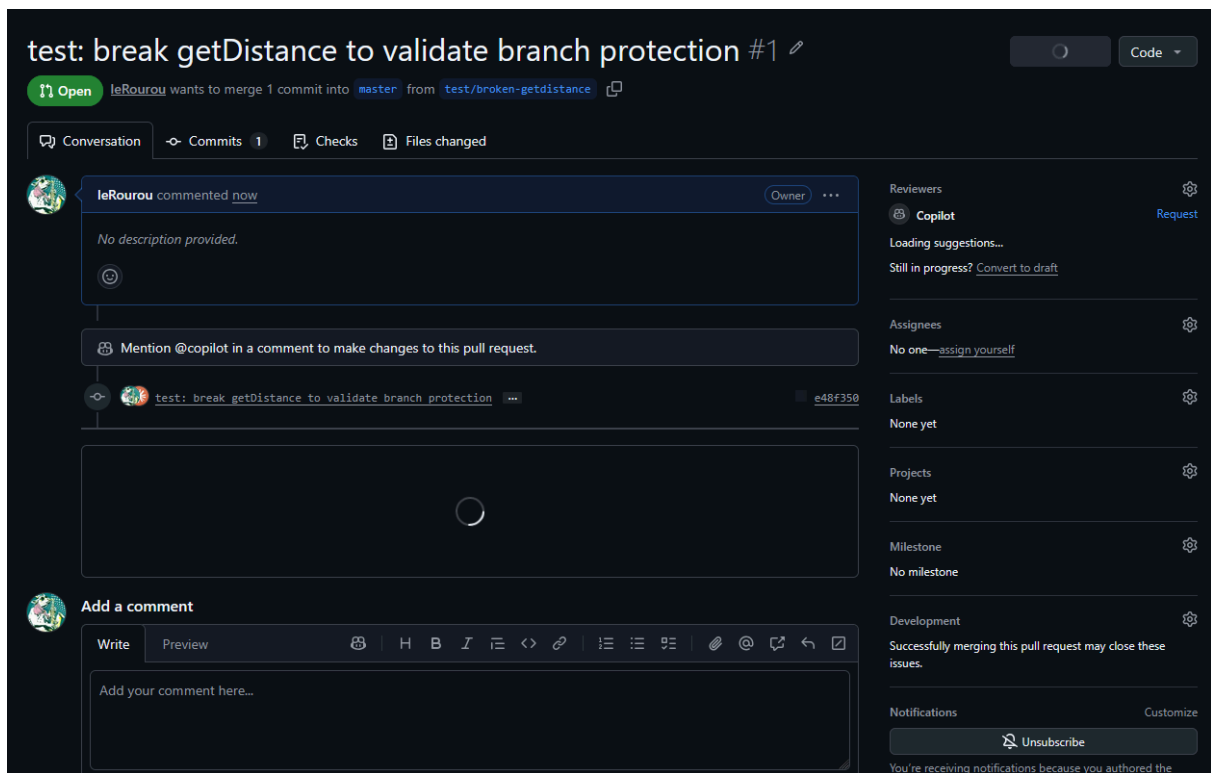
Branch targeting criteria

Add target

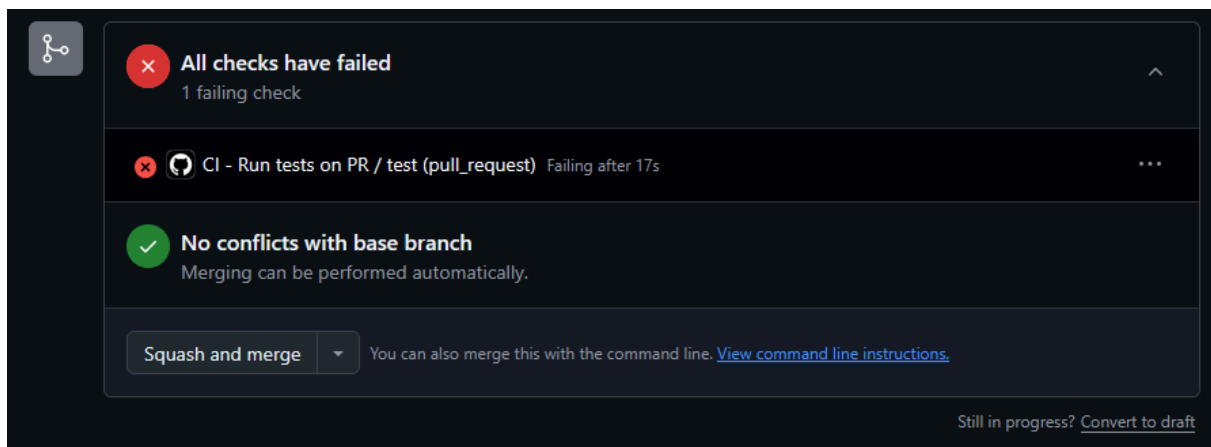
+ master

Applies to 1 target: master.

On créé ensuite la PR :



On voit bien que les tests ont échoués :



On ne peut donc pas merge la branche.

Docker Swarm

Exercice 2

On commence par créer un fichier `compose.yml` avec un manager et 3 nodes :

```

version: "3.8"

Run All Services
services:
  Run Service
  manager:
    image: docker:dind
    container_name: manager
    privileged: true
    hostname: manager
    networks:
      - swarm-net

  Run Service
  node1:
    image: docker:dind
    container_name: node1
    privileged: true
    hostname: node1
    networks:
      - swarm-net

  Run Service
  node2:
    image: docker:dind
    container_name: node2
    privileged: true
    hostname: node2
    networks:
      - swarm-net

  Run Service
  node3:
    image: docker:dind
    container_name: node3
    privileged: true
    hostname: node3
    networks:
      - swarm-net

networks:
  swarm-net:

```

Après un `docker ps`, on voit les 4 containers :

4aabb889459	docker:dind	"dockerd-entrypoint..."	7 weeks ago	Up 4 minutes	2375-2376/tcp	node2
cbb0e42eb874	docker:dind	"dockerd-entrypoint..."	7 weeks ago	Up 4 minutes	2375-2376/tcp	node1
b7151e9f403f	docker:dind	"dockerd-entrypoint..."	7 weeks ago	Up 4 minutes	2375-2376/tcp	manager
98a2b0f84f71	docker:dind	"dockerd-entrypoint..."	7 weeks ago	Up 4 minutes	2375-2376/tcp	node3

On initialise swarm sur le container `manager` : `docker exec manager docker swarm init`

On récupère ensuite le token avec la commande `docker exec manager docker swarm join-token worker`

```
rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec manager docker
swarm join-token worker
To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-6cyjbgh9zyo2jghvz4bjsgrwwjy6
nqmfvv98vjbgbysq4ug2-3axjq8xm3r0gc22dr0qigl1hu 172.23.0.5:2377
```

On ajoute les containers `node1`, `node2` et `node3` en utilisant le token : `docker exec <container> docker swarm join --token <token> manager:2377`

```
rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec node1 docker s
warm join --token SWMTKN-1-6cyjbgh9zyo2jghvz4bjsgrwwjy6nqmfvv98vjb
byssq4ug2-3axjq8xm3r0gc22dr0qigl1hu manager:2377
This node joined a swarm as a worker.
rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec node2 docker s
warm join --token SWMTKN-1-6cyjbgh9zyo2jghvz4bjsgrwwjy6nqmfvv98vjb
byssq4ug2-3axjq8xm3r0gc22dr0qigl1hu manager:2377
This node joined a swarm as a worker.
rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec node3 docker s
warm join --token SWMTKN-1-6cyjbgh9zyo2jghvz4bjsgrwwjy6nqmfvv98vjb
byssq4ug2-3axjq8xm3r0gc22dr0qigl1hu manager:2377
This node joined a swarm as a worker.
```

On vérifie enfin que les 3 noeuds ont bien été ajoutés : `docker exec manager docker node ls`

```
rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec manager docker
node ls
ID                HOSTNAME        STATUS      AVAILABILITY  MANAGER STATUS      ENGINE VERSION
iz1ypcmu1h1kvk6t220s1f035 * manager        Ready       Active         -                   29.3.1
i8vbc87wj4uhhmyvf1lth3rvm node1          Ready       Active         -                   29.3.1
ir3mid6g9cq602ar8lu4i1i6s node2          Ready       Active         -                   29.3.1
k5h0pxdyndand9jeqe44afta2 node3          Ready       Active         -                   29.3.1
```

Exercice 3

On crée un fichier `hello-world.compose.yml` et on le copie sur le container `manager` :

```

1  version: "3.8"
2
3  services:
4    · hello:
5      · · · image: hello-world
6      · · · deploy:
7        · · · · replicas: 2

```

Puis on déploie la stack sur le Docker Swarm : `docker stack deploy -c hello-world.compose.yml hello-world-stack`

```

rourou@LAPTOP-U6739NS7:~/esgi/swarm-dind$ docker exec -it manager docker stack deploy -c /hello-world.compose.yml hello-world-stack
Since --detach=false was not specified, tasks will be created in the background.
In a future release, --detach=false will become the default.
Creating network hello-world-stack_default
Creating service hello-world-stack_hello

```

Après avoir effectué un `docker ps` sur chacun des containers; on en déduit que Swarm distribue les 2 réplicas sur les nœuds disponibles. Mais comme `hello-world` s'arrête immédiatement après exécution, Swarm leredémarre en boucle sur différents nœuds pensant que c'est un crash.

On peut ensuite s'amuser à modifier la clause `deploy`. Par exemple, avec une contrainte `node.role == manager` : le service tourne uniquement sur le manager, les workers sont ignorés.

Ou encore, en spécifiant le mode global (`mode: global`), Swarm déploie automatiquement 1 container sur chaque nœud sans spécifier de nombre de réplica.

Exercice 4

Pour lancer avec 3 containers noeuds, on peut utiliser l'option `scale` de la commande Docker; par exemple `docker compose up -d --scale node=3` pour lancer avec 3 containers.

```
● rourou@LAPTOP-U6739NS7:~/esgi/esgi-2604-ansible$ docker compose up -d --scale node=3
[+] Building 17.7s (9/9) FINISHED
=> [esgi-2604-ansible-node internal] load build definition from Dockerfile 0.2s
=> => transferring dockerfile: 195B 0.0s
=> [esgi-2604-ansible-manager internal] load build definition from Dockerf 0.2s
=> => transferring dockerfile: 195B 0.0s
=> [esgi-2604-ansible-node internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [esgi-2604-ansible-manager internal] load .dockerignore 0.1s
=> => transferring context: 2B 0.0s
=> [esgi-2604-ansible-manager internal] load metadata for docker.io/librar 0.0s
=> [esgi-2604-ansible-manager 1/3] FROM docker.io/library/docker:dind 1.1s
=> [esgi-2604-ansible-node 2/3] RUN apk add --update --no-cache python3 py 9.9s
=> [esgi-2604-ansible-manager 3/3] RUN apk add sudo 5.8s
=> [esgi-2604-ansible-node] exporting to image 1.0s
=> => exporting layers 0.8s
=> => writing image sha256:bc10476b451f66acd8ea616ea73f8691b8e2c2049b7b136 0.0s
=> => naming to docker.io/library/esgi-2604-ansible-manager 0.0s
=> => writing image sha256:847b0c00ba7b719b68fe1e63f2a8752aa168755b78b2984 0.0s
=> => naming to docker.io/library/esgi-2604-ansible-node 0.0s
[+] Running 5/5
:: Network esgi-2604-ansible_default Created 0.2s
:: Container esgi-2604-ansible-node-3 Started 4.3s
:: Container esgi-2604-ansible-manager-1 Started
:: Container esgi-2604-ansible-node-1 Started
:: Container esgi-2604-ansible-node-2 Started
```

On peut ensuite lancer `docker ps` et vérifier qu'il y a bien 4 containers.

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
19cd947709b5	esgi-2604-ansible-node	"dockerd-entrypoint...."	41 seconds ago	Up 39 seconds
2375-2376/tcp		esgi-2604-ansible-node-2		
bd52e23f697c	esgi-2604-ansible-node	"dockerd-entrypoint...."	42 seconds ago	Up 37 seconds
2375-2376/tcp		esgi-2604-ansible-node-3		
b83d97d830b0	esgi-2604-ansible-node	"dockerd-entrypoint...."	42 seconds ago	Up 35 seconds
2375-2376/tcp		esgi-2604-ansible-node-1		
72879c36a13a	esgi-2604-ansible-manager	"dockerd-entrypoint...."	42 seconds ago	Up 39 seconds
2375-2376/tcp		esgi-2604-ansible-manager-1		

Le playbook Ansible initialise un cluster Docker Swarm en désignant un manager et en y rattachant automatiquement 3 workers :

```

TASK [Join swarm as worker] *****
changed: [esgi-2604-ansible-node-1]
changed: [esgi-2604-ansible-node-2]
changed: [esgi-2604-ansible-node-3]

PLAY RECAP *****
esgi-2604-ansible-manager-1 : ok=6    changed=3    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0
esgi-2604-ansible-node-1   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0
esgi-2604-ansible-node-2   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0
esgi-2604-ansible-node-3   : ok=2    changed=1    unreachable=0    failed=0    skipped=0    rescued=0
    ignored=0

```

Lorsque l'on lance le même playbook, sur les mêmes machines playbook se relance sans erreur grâce aux `failed_when`, mais affiche `changed` sur toutes les tâches au lieu de `ok` car les modules `command`/`shell` ne vérifient jamais l'état existant.

Exercice 5

On ajoute un quatrième noeud : `docker compose up --scale node=4 -d` et on la rajoute à l'`inventory`.

On vérifie ensuite qu'il est bien présent dans le cluster :

```

rourou@LAPTOP-U6739NS7:~/esgi/esgi-2604-ansible$ docker exec esgi-2604-ansible-manager-1 docker node ls

```

ID	HOSTNAME	STATUS	AVAILABILITY	MANAGER STATUS	ENGINE VERSION
hijlks1opzui1r6dozxadqwd *	0f8101e087e2	Ready	Active	Leader	29.3.1
w64mj90dr4hyve3j5vfp9rocm	91d1c1fef006	Ready	Active		29.3.1
cqrfrq30jogcmi0gjbuuqbbe1	93502bdf3644	Ready	Active		29.3.1
9yctdglkhwsdnu0jiu9fal1hq	ae2e851c72b	Ready	Active		29.3.1
0m59uct3vs06096ffzevljc9o	ddd18acaa2cc	Ready	Active		29.3.1

On devra modifier `inventory.ini` : remplacer les noms de conteneurs par les IPs des VMs et

`ansible_connection=community.docker.docker` par `SSH + user + clé privée` ainsi que `playbook` : ajouter un play d'installation de Docker au début, puisque les VMs n'ont pas Docker préinstallé.

Terraform et Ansible sont souvent utilisés ensemble : Terraform crée l'infrastructure (serveurs, réseaux, bases de données) tandis qu'Ansible la configure une fois en place.

Traefik

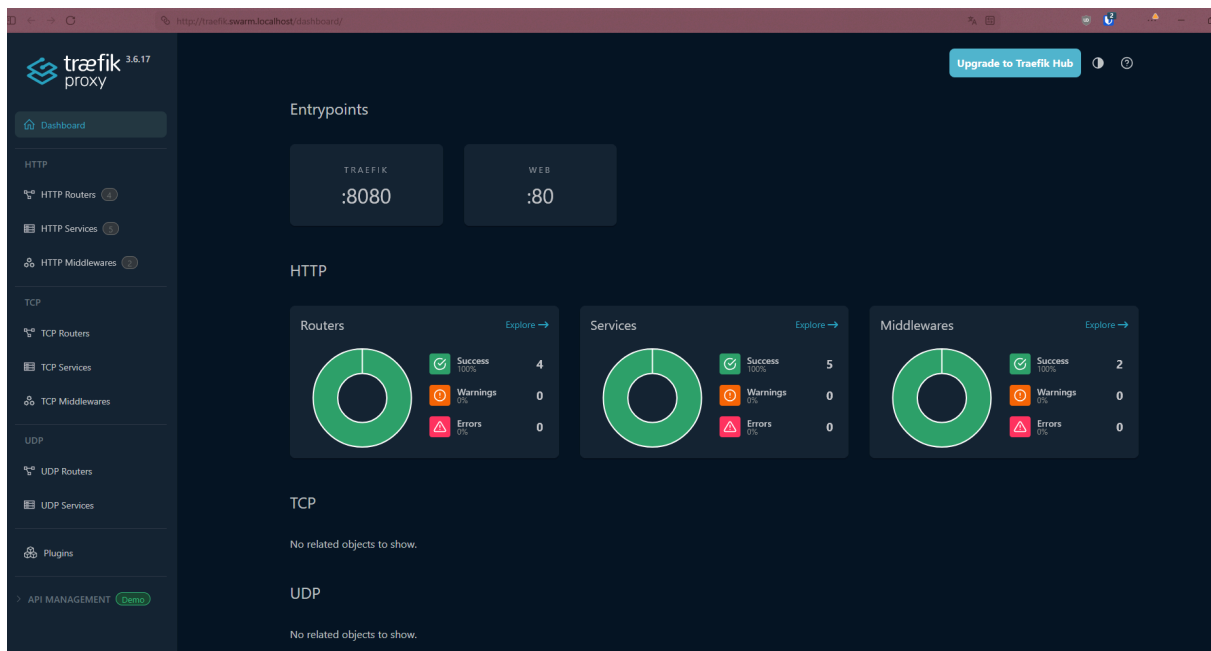
On va mettre en place Traefik, puis vérifier que l'on a bien accès à `traefik.swarm.localhost` et `whoami.swarm.localhost`.

`whoami.swarm.localhost` :

```
http://whoami.swarm.localhost

Hostname: 62937d7ace30
IP: 127.0.0.1
IP: ::1
IP: 10.0.1.6
IP: 172.18.0.3
RemoteAddr: 10.0.1.3:37070
GET / HTTP/1.1
Host: whoami.swarm.localhost
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:150.0) Gecko/20100101 Firefox/150.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Encoding: gzip, deflate, br, zstd
Accept-Language: fr,fr-FR;q=0.9,en-US;q=0.8,en;q=0.7
Priority: u=0, i
Sec-Fetch-Dest: document
Sec-Fetch-Mode: navigate
Sec-Fetch-Site: none
Sec-Gpc: 1
Upgrade-Insecure-Requests: 1
X-Forwarded-For: 10.0.0.2
X-Forwarded-Host: whoami.swarm.localhost
X-Forwarded-Port: 80
X-Forwarded-Proto: http
X-Forwarded-Server: f2f294807c0b
X-Real-Ip: 10.0.0.2
```

`traefik.swarm.localhost` :



Après avoir cloné l'`example-voting-app`, on modifie la stack :

```

1  # this file is meant for Docker Swarm stacks only
2  # trying it in compose will fail because of multiple replicas trying to bind to the same po
3  # Swarm currently does not support Compose Spec, so we'll pin to the older version 3.9
4
5  version: "3.9"
6
7  services:
8
9+   traefik:
10+     image: traefik:v3.0
11+     command:
12+       - "--providers.swarm=true"
13+       - "--providers.swarm.exposedbydefault=false"
14+       - "--entrypoints.web.address=:80"
15+     ports:
16+       - "80:80"
17+     volumes:
18+       - /var/run/docker.sock:/var/run/docker.sock:ro
19+     networks:
20+       - proxy
21+     deploy:
22+       placement:
23+         constraints:
24+           - node.role == manager
25+
26   redis:
27     image: redis:alpine
28     networks:
29     - frontend

```

```

43     ... networks:
44     ...     ... frontend
45+     ...     - proxy
46     ... deploy:
47     ...     replicas: 2
48+     ...     labels:
49+     ...         - "traefik.enable=true"
50+     ...         - "traefik.http.routers.vote.rule=Host(`vote.local`)"
51+     ...         - "traefik.http.routers.vote.entrypoints=web"
52+     ...         - "traefik.http.services.vote.loadbalancer.server.port=80"
53
54     ... result:
55     ...     image: dockersamples/examplevotingapp_result
56
57     ... networks:
58+     ...     - proxy
59+     ...     deploy:
60+     ...         replicas: 1
61+     ...         labels:
62+     ...             - "traefik.enable=true"
63+     ...             - "traefik.http.routers.result.rule=Host(`result.local`)"
64+     ...             - "traefik.http.routers.result.entrypoints=web"
65+     ...             - "traefik.http.services.result.loadbalancer.server.port=80"
66
67     ... worker:
68     ...     image: dockersamples/examplevotingapp_worker
69     ...     networks:
70     ...         ... frontend
71     ...         ... backend
72     ...     deploy:
73     ...         replicas: 2
74
75     networks:
76     ... frontend:
77     ... backend:
78+     proxy:
79
80     volumes:
81     ... db-data:
82

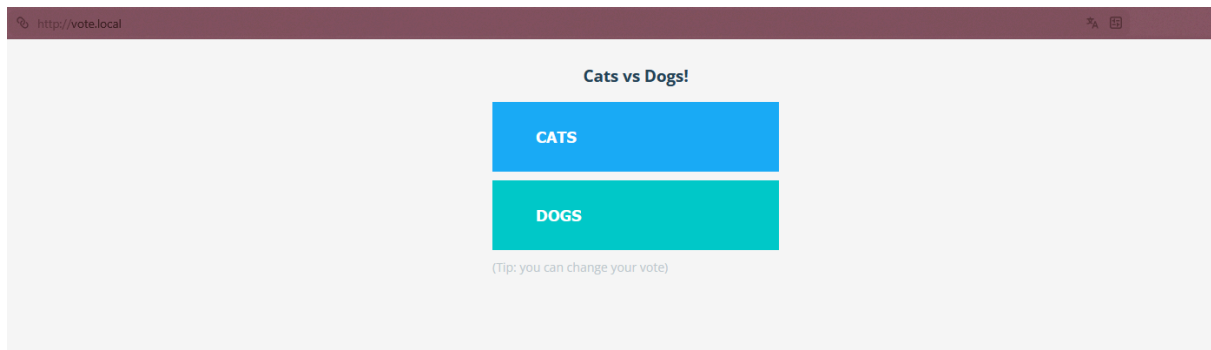
```

Puisque l'on est sous WSL, on ajoute au Path les hosts grâce au PowerShell:

```

Add-Content -Path "C:\Windows\System32\drivers\etc\hosts" -Value "127.0.0.1`tvote.local"
Add-Content -Path "C:\Windows\System32\drivers\etc\hosts" -Value
"127.0.0.1`tresult.local"

```

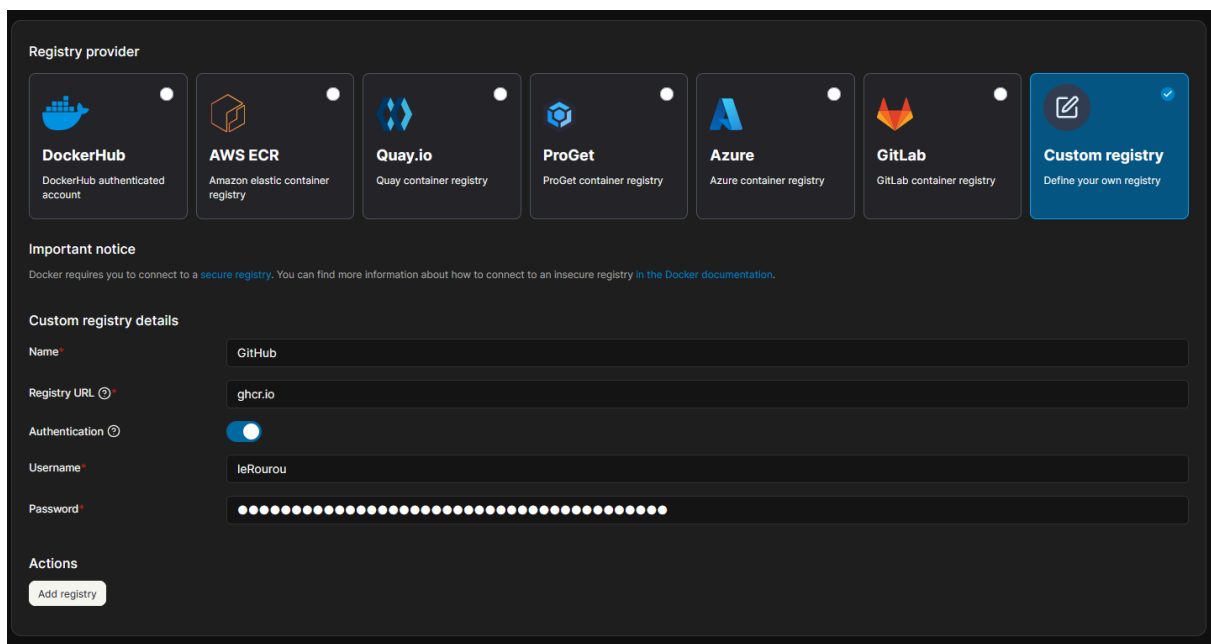


CD

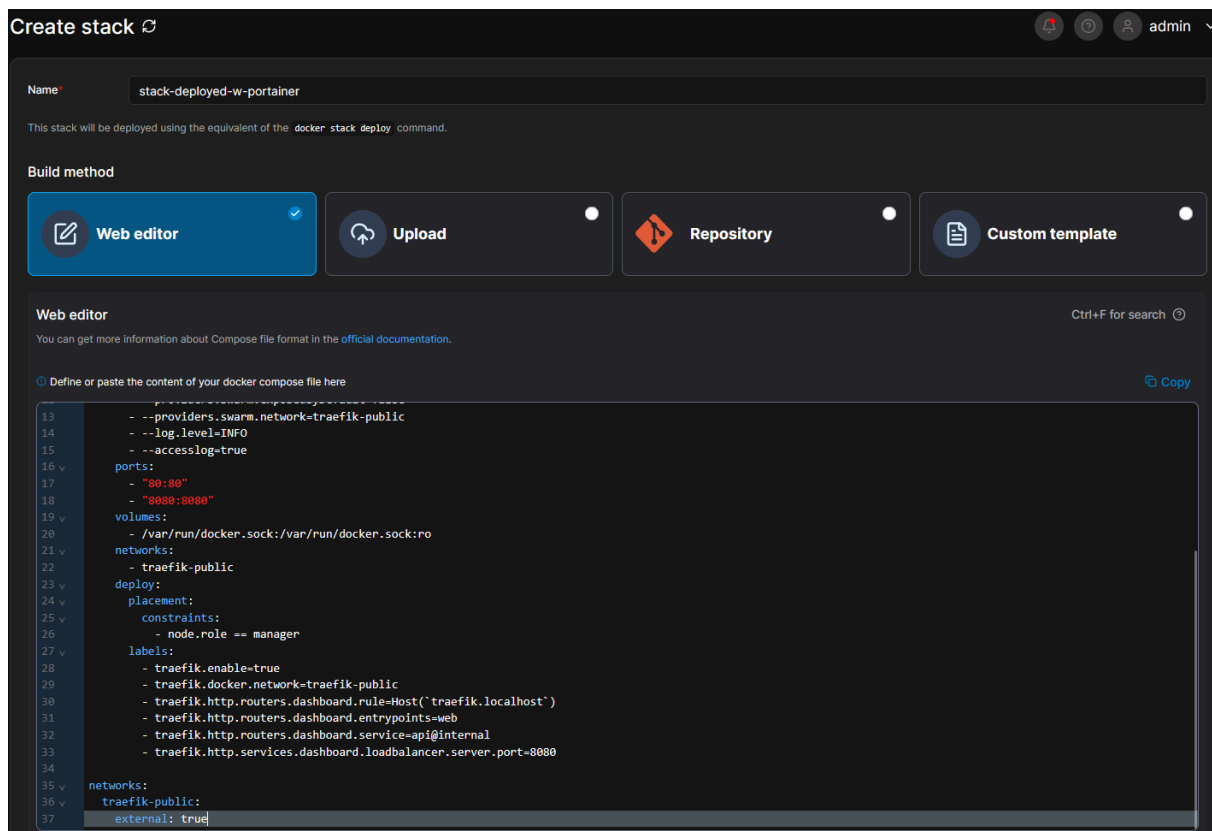
On commence par ajouter un endpoint `/bonjour` qui renvoie uniquement *"Bonjour !"*

```
apiRouter.get("/bonjour", (_, res) => {  
  res.send("Bonjour !");  
});
```

Ensuite, on va ajouter un nouveau registry à Portainer. On va ajouter le registry de GitHub, là où sont stockées nos images: `ghcr.io`. On aura préalablement généré un token avec les droits en lecture.



Puis on déploie notre nouvelle stack sur l'environnement Swarm.



Voilà la stack qui sera déployée :

```
version: "3.8"
```

```
services:
```

```
traefik:
```

```
image: traefik:v3.1
```

```
command:
```

```

- --api.dashboard=true
- --api.insecure=true
- --entrypoints.web.address=:80
- --providers.swarm=true
- --providers.swarm.endpoint=unix:///var/run/docker.s

```

```
ock
```

```

- --providers.swarm.exposedByDefault=false
- --providers.swarm.network=favorite-places_public
- --log.level=INFO
- --accesslog=true

```

```
ports:
```

```
- "80:80"
```

```

volumes:
  - /var/run/docker.sock:/var/run/docker.sock:ro
networks:
  - public
deploy:
  placement:
    constraints:
      - node.role == manager
  labels:
    - traefik.enable=true
    - traefik.docker.network=favorite-places_public
    - traefik.http.routers.dashboard.rule=Host(`traefik.localhost`)
    - traefik.http.routers.dashboard.entrypoints=web
    - traefik.http.routers.dashboard.service=api@internal
    - traefik.http.services.dashboard.loadbalancer.server.port=8080

db:
  image: postgres:17-alpine
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: supersecret
    POSTGRES_DB: postgres
  volumes:
    - db_data:/var/lib/postgresql/data
  networks:
    - internal
  deploy:
    replicas: 1
    placement:
      constraints:
        - node.role == manager
    restart_policy:
      condition: on-failure

server:

```

```

    image: ghcr.io/lerourou/my-favorite-places-server:lates
t
    environment:
      DB_HOST: db
      DB_USER: postgres
      DB_PASSWORD: supersecret
      DB_NAME: postgres
    networks:
      - internal
      - public
    deploy:
      replicas: 1
      update_config:
        order: start-first
        failure_action: rollback
      restart_policy:
        condition: on-failure
    labels:
      - traefik.enable=true
      - traefik.docker.network=favorite-places_public
      - traefik.http.routers.server.rule=Host(`api.localh
ost`)
      - traefik.http.routers.server.entrypoints=web
      - traefik.http.services.server.loadbalancer.server.
port=3000

    client:
      image: ghcr.io/lerourou/my-favorite-places-client:lates
t
      networks:
        - public
      deploy:
        replicas: 1
        update_config:
          order: start-first
          failure_action: rollback
        restart_policy:
          condition: on-failure

```

```

labels:
  - traefik.enable=true
  - traefik.docker.network=favorite-places_public
  - traefik.http.routers.client.rule=Host(`app.localh
ost`)
  - traefik.http.routers.client.entrypoints=web
  - traefik.http.services.client.loadbalancer.server.
port=80

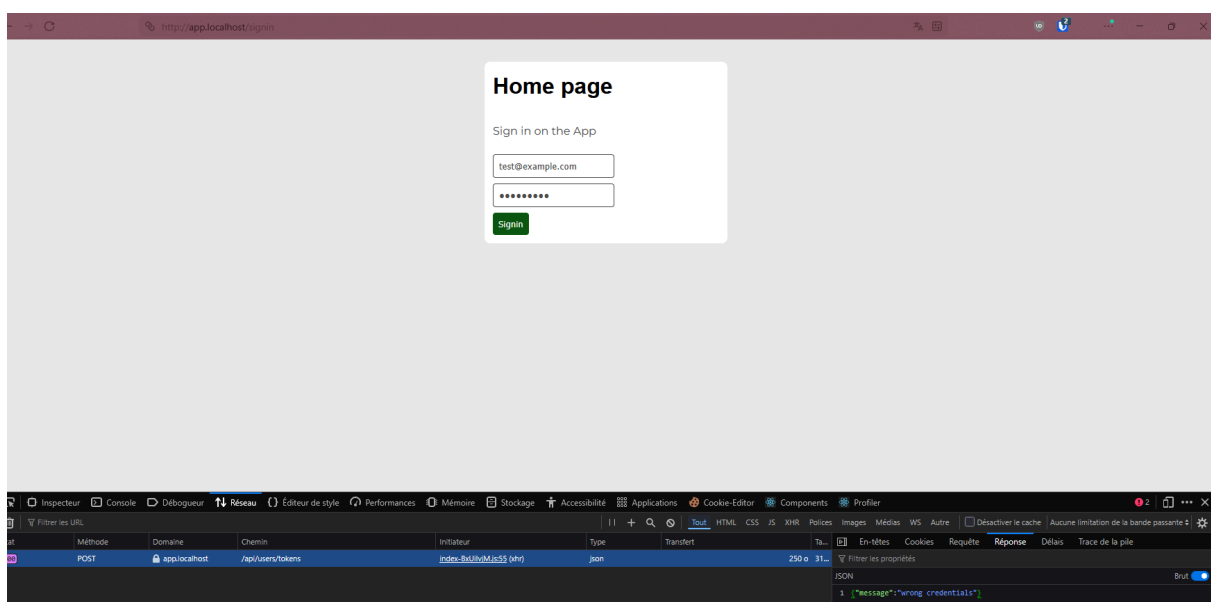
volumes:
  db_data:

networks:
  public:
    driver: overlay
  internal:
    driver: overlay

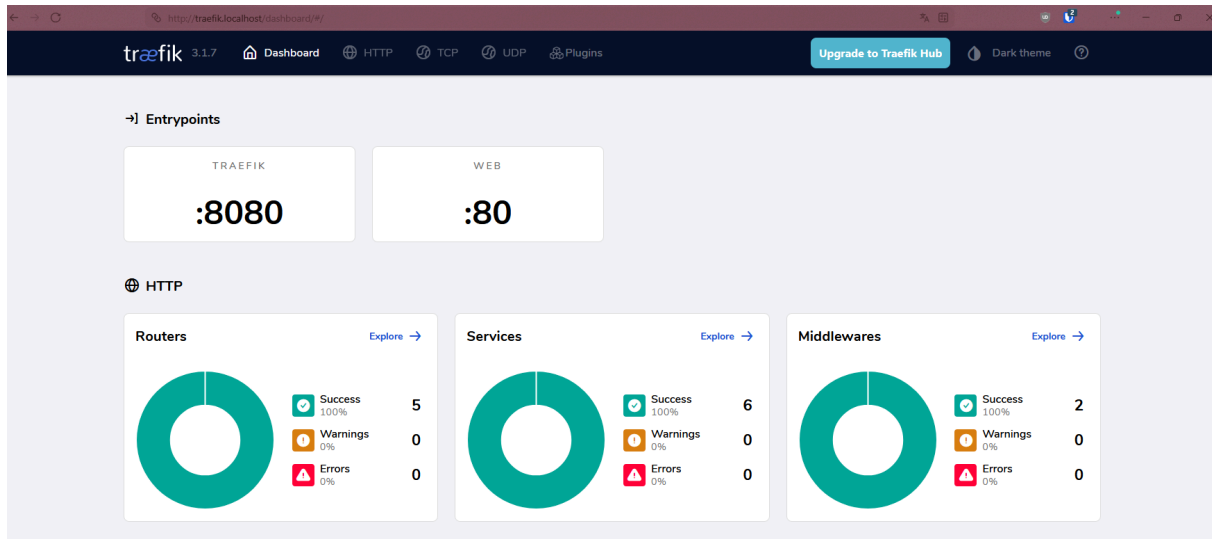
```

Il s'agit au final d'une stack Swarm qui démarre Traefik comme reverse proxy sur le port 80, expose le client sur `app.localhost`, le serveur sur `api.localhost` et le dashboard Traefik sur `traefik.localhost`, le tout connecté à Postgres via un réseau interne privé.

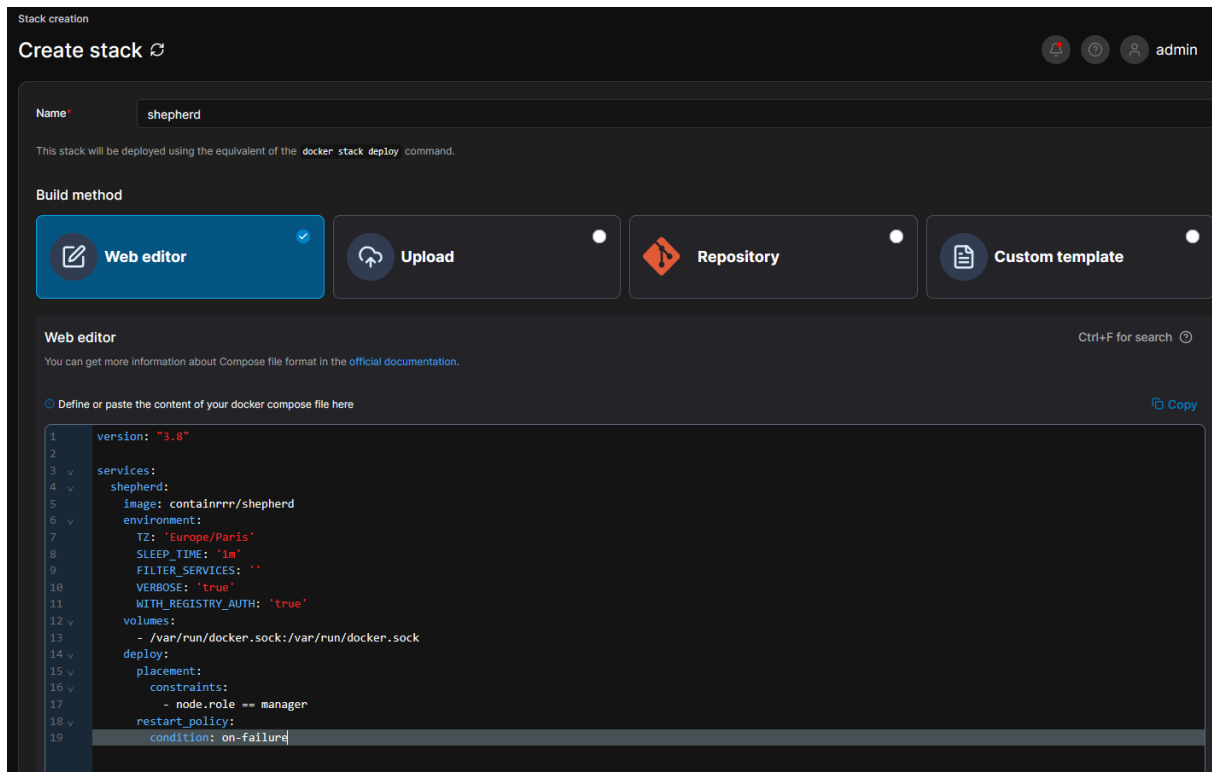
Ce que l'on voit sur `app.localhost` :



Ce que l'on voit sur `traefik.localhost` :



Nous allons maintenant déployer une stack Shepherd, en suivant les bonnes pratiques spécifiées dans le [README.MD](#) :



En allant voir les logs du service Shepherd :

```

Wed May 27 11:16:39 CEST 2026 Timezone set to Europe/Paris
Wed May 27 11:16:39 CEST 2026 Enabling synchronous service updates
Wed May 27 11:16:39 CEST 2026 Send registry authentication details to swarm agents
Wed May 27 11:16:41 CEST 2026 Trying to update service docker-swarm-mfp_client with image ghcr.io/lerourou/my-favorite-places-client:latest
Wed May 27 11:16:49 CEST 2026 No updates to service docker-swarm-mfp_client!
toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
Wed May 27 11:16:52 CEST 2026 Error updating service docker-swarm-mfp_db! Image postgres:17-alpine does not exist or it is not available
Wed May 27 11:16:55 CEST 2026 Trying to update service docker-swarm-mfp_server with image ghcr.io/lerourou/my-favorite-places-server:latest
Wed May 27 11:17:02 CEST 2026 No updates to service docker-swarm-mfp_server!
toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
Wed May 27 11:17:04 CEST 2026 Error updating service docker-swarm-mfp_traefik! Image traefik:v3.1 does not exist or it is not available
toomanyrequests: You have reached your unauthenticated pull rate limit. https://www.docker.com/increase-rate-limit
Wed May 27 11:17:07 CEST 2026 Error updating service shepherd_shepherd! Image containrrr/shepherd:latest does not exist or it is not available
Wed May 27 11:17:07 CEST 2026 Sleeping 1m before next update

```

On voit que le service effectue du polling sur les différentes images pull par la stack que nous avons déployée juste avant. Il cherche les dernières mises à jours.

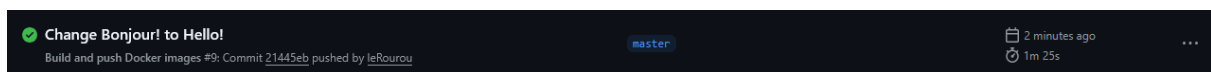
Pour tester si cela fonctionne bien, nous allons modifier notre endpoint `/bonjour`

```

server/src/router.ts
@@ -5,7 +5,7 @@ import addressesRouter from "../controllers/Addresses";
5  const apiRouter = Router();
6
7  apiRouter.get("/bonjour", (_, res) => {
8    - res.send("Bonjour !");
8    + res.send("Hello !");
9  });
10
11 apiRouter.use("/users", usersRouter);

```

Cela a d'abord pour effet de trigger notre Workflow :



Et cela a enfin pour effet à ce que notre image soit mise à jour automatiquement par Shepherd.

```

Wed May 27 11:44:58 CEST 2026 Trying to update service docker-swarm-mfp_server with image ghcr.io/lerourou/my-favorite-places-server:latest
Wed May 27 11:45:46 CEST 2026 Service docker-swarm-mfp_server was updated!

```

Je pense que si cela ne fonctionne plus ensuite, c'est parce qu'en Docker Swarm, `depends_on` est ignoré, donc le `server` démarre en parallèle de `db` et plante car Postgres n'est pas encore prête à accepter des connexions.